21. Tasarım Kalıpları (Design Patterns): Factory, Strategy, Observer Pratikte

Tekerleği Yeniden İcat Etmeden Endüstri Standartlarında Kod Yazmak
Bölüm 4 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Gang of Four tasarım kalıpları 1994 yılında yazıldı. 2026'da AI çağında Factory, Strategy veya Observer kalıpları hâlâ geçerli mi?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Tasarım kalıpları ezberlenecek formüller değil; yazılımcıların ve yapay zekanın ortak kelime dağarcığıdır (Vocabulary). - 📌 AI'a 'Burada 10 tane if-else yazmak yerine Strategy Pattern kullan' demek, tek kelimeyle mükemmel mimari kurdurur. - 📌 Observer deseni, günümüzün modern olay tabanlı (Event-Driven) sistemlerinin ve bildirim mekanizmalarının temelidir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir restorana gittiğinizi ve garsona sipariş verdiğinizi hayal edin.

Garsona her seferinde: *"Önce bir tabağı dolaptan çıkar, sonra içine 200 gram un koy, fırını 180 dereceye ayarla, fırına sür ve piştikten sonra masama getir"* diyor musunuz? Hayır! Sadece menüden: *"Bir pizza Margherita lütfen"* dersiniz. Mutfak bunu nasıl üreteceğini zaten bilir. İşte bu **Factory (Fabrika)** desenidir!

Peki pizzayı nasıl ödeyeceksiniz? Nakit mi, kredi kartı mı, kripto ile mi? Kasiyer ödeme yöntemine göre pos cihazını uzatır veya nakit alır. İşte bu **Strategy (Strateji)** desenidir!

Pizzanız hazır olduğunda masadaki çağrı cihazı titrer ve size haber verir. İşte bu da **Observer (Gözlemci)** desenidir!

Tasarım kalıpları hayatın içindeki standart çözümlerdir. Kodda da aynısı geçerlidir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Çok Kullanılan 3 Tasarım Kalıbı:

```
1. FACTORY PATTERN (Yaratımsal):
   İstemci nesnenin nasıl üretildiğini bilmez; sadece 'Bana X tipinde bir nesne ver'
der.
   Kullanım: Farklı veritabanı sürücüleri, ödeme sağlayıcıları oluşturma.

2. STRATEGY PATTERN (Davranışsal):
   Çalışma zamanında (runtime) algoritmanın veya davranışın değiştirilebilmesini
sağlar.
   Kullanım: Farklı kargo hesaplama algoritmaları, indirim kuponu stratejileri.

3. OBSERVER PATTERN (Davranışsal):
   Bir nesnede durum değiştiğinde, ona abone olan tüm diğer nesneleri otomatik
haberdar eder.
   Kullanım: Event Emitter'lar, Webhook bildirimleri, WebSocket yayınları.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Yazılım Tasarım Kalıpları Mimarı
Projemizde kullanıcılar 3 farklı yöntemle bildirim alabilmektedir:
- SMS (Twilio)
- E-posta (SendGrid)
- Push Notification (Firebase)

## GÖREV:
Bu yapıyı 'Strategy Pattern' kullanarak TypeScript ile kur.
1. Ortak bir `NotificationStrategy` interface'i tanımla (`send(userId, message)`).
2. Her kanal için ayrı bir somut strateji sınıfı yaz.
3. Bir `NotificationContext` sınıfı yazarak çalışma zamanında stratejinin dinamik
olarak değiştirilebilmesini sağla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü: 20 tane if-else ile spagetti kod
export async function sendNotificationSpaghetti(type: string, userId: string, msg:
string) {
  if (type === "SMS") {
    // 30 satır Twilio kodu...
  } else if (type === "EMAIL") {
    // 30 satır SendGrid kodu...
  } else if (type === "PUSH") {
    // 30 satır Firebase kodu...
  } else {
    throw new Error("Bilinmeyen bildirim türü");
  }
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
// Strategy Pattern ile Zarif ve Genişletilebilir Mimari

// 1. Strateji Arayüzü (Contract)
export interface NotificationStrategy {
  send(userId: string, message: string): Promise<boolean>;
}

// 2. Somut Stratejiler (Concrete Strategies)
export class EmailNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[SendGrid] ${userId} kullanıcısına e-posta gönderildi: ${message}
`);
    return true;
  }
}

export class SmsNotificationStrategy implements NotificationStrategy {
  async send(userId: string, message: string): Promise<boolean> {
    console.log(`[Twilio] ${userId} kullanıcısına SMS atıldı: ${message}`);
    return true;
  }
}

// 3. Bağlam Yöneticisi (Context)
export class NotificationService {
  private strategy: NotificationStrategy;

  constructor(initialStrategy: NotificationStrategy) {
    this.strategy = initialStrategy;
  }

  setStrategy(newStrategy: NotificationStrategy): void {
    this.strategy = newStrategy;
  }

  async dispatch(userId: string, message: string): Promise<boolean> {
    return await this.strategy.send(userId, message);
  }
}
```

💡 **Kod Analizi:** Artık sisteme ‘WhatsApp’ veya ‘Slack’ bildirimi eklemek istediğimizde mevcut koda tek bir satır dahi dokunmayız (Open/Closed Principle). Sadece yeni bir `WhatsAppStrategy` sınıfı ekleriz.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Basit 2 durumlu bir kontrol için devasa Factory veya Strategy sınıfları kurmak.

  - **Çözüm:** Eğer durum sayısı 2'yi geçmeyecekse basit bir `if/else` yeterlidir; kalıpları karmaşıklık arttığında devreye sokun.
- **Tuzak:** Observer deseninde abonelikten çıkmayı (unsubscribe) unutup bellek sızıntısına (memory leak) yol açmak.
  - **Çözüm:** Bileşenler yok edildiğinde (unmount) event listener'ları mutlaka temizleyin.
- **Tuzak:** Kalıpların isimlerini projedeki isimlendirmelere yansıtmamak.
  - **Çözüm:** Sınıf isimlerine `PaymentStrategy`, `LoggerFactory` gibi açık ekler verin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 21: Tasarım Kalıpları - Factory, Strategy ve Observer*

- 🖥️ **Slayt 1:** Tasarım kalıpları neden yapay zeka ile geliştiricinin ortak dilidir?
- 🖥️ **Slayt 2:** If-Else canavarından Strategy Pattern ile kurtulmak.
- 🖥️ **Slayt 3:** Factory Pattern ile nesne yaratımını soyutlamak.
- 🖥️ **Slayt 4:** Observer Pattern: Event-Driven mimarinin temeli.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir e-ticaret kargo hesaplayıcısı yazın. 'Aras', 'Yurtiçi' ve 'DHL' için 3 farklı strateji sınıfı kurun. Kullanıcı kargo firmasını seçtiğinde runtime'da stratejinin nasıl değiştiğini ve doğru ücreti hesapladığını konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde en az 3 farklı `else if` içeren bir kod bloğu bulun ve bunu Strategy Pattern ile refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Strategy Pattern'in temel amacı nedir? - A) Veritabanındaki tüm tabloları silmek - B) Birbirinin yerine geçebilen algoritmalar ailesi tanımlayarak, algoritmayı onu kullanan istemciden bağımsız şekilde çalışma zamanında değiştirebilmek - C) Sadece HTML dosyalarını sıkıştırmak - D) Bilgisayarın pil ömrünü uzatmak > **Doğru Yanıt:** B > **Açıklama:** Strategy Pattern, çalışma zamanında farklı algoritmaların (örn: ödeme veya sıralama yöntemleri) dinamik olarak birbirinin yerine kullanılmasını sağlar.

Soru 2: Observer Pattern'de bildirimleri dinleyen taraflara ne ad verilir? - A) Subscriber (Abone / Gözlemci) - B) Compiler - C) Terminal - D) Router > **Doğru Yanıt:** A > **Açıklama:** Observer deseninde durum yayınlayan nesne Subject (Yayımcı), bu durumu dinleyen nesneler ise Observer / Subscriber (Abone) olarak adlandırılır.